

AlerjiTakvim

Back-End API Dokümantasyonu

OpenAPI 3.0 Uyumlu REST API Referansı

Versiyon: 2.0 | Haziran 2026

Base URL: <http://localhost:3001>

Bu dokümantasyon AlerjiTakvim projesinin tüm backend API endpointlerini, parametrelerini, yanıt yapılarını ve frontend entegrasyon detaylarını içermektedir.

Genel Bilgiler

Alan	Değer
Base URL (geliştirme)	http://localhost:3001
Base URL (üretim)	https://api.alerjitakvim.com (örnek)
Protokol	HTTPS (üretim) / HTTP (geliştirme)
İçerik Tipi	application/json (istek ve yanıt)
Kimlik Doğrulama	Bearer Token (JWT HS256, 7 gün geçerli)
Token Formatı	Authorization: Bearer <JWT_TOKEN>
Rate Limit (genel)	300 istek / 15 dakika / IP adresi
Rate Limit (auth)	20 istek / 15 dakika / IP adresi
API Versiyon	v2.0 (versiyon URL prefix yok)

Tablo G.1 — Genel API yapılandırması

Kimlik Doğrulama Akışı

- POST /api/register veya POST /api/login ile JWT token alın.
- Her korumalı istek için Authorization başlığına "Bearer {token}" ekleyin.
- Token süresi dolduğunda (7 gün) sistem 401 döner; kullanıcıyı tekrar giriş ekranına yönlendirin.

Standart Hata Yanıt Yapısı

```
{ "error": "İnsan okunabilir mesaj", "code": "MAKINA_KODU", "details": { } }
```

HTTP Kodu	code	Açıklama
400	VALIDATION_ERROR / MISSING_FIELD	Ekli veya geçersiz giriş
401	UNAUTHORIZED	Geçersiz veya süresi dolmuş JWT
402	PAYMENT_FAILED	Ödeme doğrulanamadı
403	FORBIDDEN	Yetkisiz erişim (başkasının kaynağı)
404	NOT_FOUND	Kaynak bulunamadı
429	RATE_LIMITED / QUOTA_EXCEEDED	Rate limit veya günlük kota aşımı
500	INTERNAL_ERROR	Beklenmeyen hata (üretimde stack trace gizli)
502	LLM_UNAVAILABLE	Tüm Groq model/anahtar kombinasyonları başarısız
503	DB_UNAVAILABLE	Veritabanı erişilemiyor (/api/health)

Tablo G.2 — Standart hata kodları

1. Kimlik Doğrulama Endpointleri

POST Kullanıcı Kaydı /api/register

Yeni bir kullanıcı hesabı oluşturur. Başarılı kayıt sonrasında otomatik giriş sağlanır ve JWT token döner.

Kimlik Doğrulama	Yok (public)	Kota	Yok (auth rate limiter: 20/15dk)	
Parametre	Yer	Tip	Zorunlu	Açıklama
name	body	string	Evet	Görünen ad (2-60 karakter)
email	body	string	Evet	Geçerli e-posta adresi (küçük harfe dönüştürülür)
password	body	string	Evet	Minimum 8 karakter
passwordConfirm	body	string	Evet	password ile eşleşmeli
HTTP	Durum	Yanıt Yapısı		
201	CREATED	{ token: string (JWT), user: { id, name, email, plan, avatar, allergies[] } }		
400	BAD_REQUEST	{ error, code: "VALIDATION_ERROR" } — eksik alan veya parola eşleşmiyor		
409	CONFLICT	{ error, code: "EMAIL_TAKEN" } — e-posta zaten kayıtlı		
429	TOO MANY	{ error, code: "RATE_LIMITED" } — 20 istek/15dk aşıldı		

Frontend Kullanımı: AuthContext.register() → api/client.js register() → localStorage.setItem("token", data.token)

POST Kullanıcı Girişi /api/login

E-posta ve şifre ile kimlik doğrular. Başarılı girişte JWT token döner. Şifre bcrypt ile doğrulanır.

Kimlik Doğrulama		Yok (public)		Kota	Yok (auth rate limiter: 20/15dk)
Parametre	Yer	Tip	Zorunlu	Açıklama	
email	body	string	Evet	Kayıtlı e-posta adresi	
password	body	string	Evet	Hesap şifresi	
HTTP	Durum	Yanıt Yapısı			
200	OK	{ token: string (JWT), user: { id, name, email, plan, avatar, allergies[] } }			
400	BAD_REQUEST	{ error, code: "MISSING_FIELD" } — alan eksik			
401	UNAUTHORIZED	{ error, code: "INVALID_CREDENTIALS" } — yanlış e-posta veya şifre			
429	TOO MANY	{ error, code: "RATE_LIMITED" }			

Frontend Kullanımı: api/client.js login() → localStorage.setItem("token", data.token) → AuthContext kullanıcıyı günceller

GET Oturum Sahibi Bilgisi /api/me

Geçerli JWT token'ına ait kullanıcı bilgilerini döner. Sayfa yenilemesinde oturum kontrolü için kullanılır.

Kimlik Doğrulama	Zorunlu (Bearer JWT)	Kota	Yok
-------------------------	----------------------	-------------	-----

HTTP	Durum	Yanıt Yapısı
200	OK	{ id, name, email, plan, plan_expires_at, avatar, allergies[], favorites[], created_at }
401	UNAUTHORIZED	{ error, code: "UNAUTHORIZED" } — geçersiz veya süresi dolmuş token

Frontend Kullanımı: AuthContext.checkAuth() → api/client.js getMe() — uygulama başlangıcında ve token yenilemesinde çağrılır

PUT Profil Güncelleme /api/profile

Kullanıcının adını, avatar emojisini ve alerjen listesini günceller. Tüm alanlar opsiyoneldir.

Kimlik Doğrulama		Zorunlu (Bearer JWT)		Kota	Yok
Parametre	Yer	Tip	Zorunlu	Açıklama	
name	body	string	Hayır	Yeni görünen ad (2-60 karakter)	
avatar	body	string	Hayır	Tek emoji (max 16 karakter)	
allergies	body	string[]	Hayır	Alerjen anahtarları (max 50 öge, ör: ["BIRCH","GRASS"])	
HTTP	Durum	Yanıt Yapısı			
200	OK	{ id, name, email, plan, avatar, allergies[], favorites[] }			
400	BAD_REQUEST	{ error, code: "VALIDATION_ERROR" } — alan tipi veya uzunluk hatası			
401	UNAUTHORIZED	Geçersiz token			

Frontend Kullanımı: DashboardProfilePage → api/client.js updateProfile({ name, avatar, allergies }) → AuthContext kullanıcıyı günceller

PUT Favori Konumlar /api/favorites

Kullanıcının favori konum listesini günceller. Mevcut liste tamamen değiştirilir (PUT semantiği).

Kimlik Doğrulama		Zorunlu (Bearer JWT)		Kota	Yok
Parametre		Yer	Tip	Zorunlu	Açıklama
favorites		body	object[]	Evet	Konum listesi (max 100). Her öge: { lat, lng, name }
HTTP	Durum	Yanıt Yapısı			
200	OK	{ favorites: [{ lat, lng, name }] }			
400	BAD_REQUEST	{ error, code: "VALIDATION_ERROR" } — dizi değil veya max 100 aşıldı			
401	UNAUTHORIZED	Geçersiz token			

Frontend Kullanımı: MapSelector favori ekle/çıkart butonu → api/client.js updateFavorites(favorites[])

DELETE Hesap Silme /api/account

Kullanıcı hesabını ve tüm ilişkili verileri kalıcı olarak siler. Ek güvenlik için şifre tekrar doğrulanır.

Kimlik Doğrulama	Zorunlu (Bearer JWT)			Kota	Yok
Parametre	Yer	Tip	Zorunlu	Açıklama	
password	body	string	Evet	Hesap şifresi (yeniden doğrulama için)	

HTTP	Durum	Yanıt Yapısı
200	OK	{ message: "Hesabınız silindi" }
400	BAD_REQUEST	{ error, code: "MISSING_FIELD" } — şifre eksik
401	UNAUTHORIZED	{ error, code: "WRONG_PASSWORD" } — şifre yanlış veya token geçersiz

Frontend Kullanımı: AccountMenu "Hesabı Sil" → onay dialog → api/client.js deleteAccount(password) → localStorage.removeItem("token")

2. Kullanım ve Kota Endpointleri

GET **Günlük Kullanım Sayaçları** /api/usage

Mevcut kullanıcı veya IP adresi için bugünkü API kullanım sayaçlarını döner. Frontend kota çubuğu bu endpoint'i kullanır.

Kimlik Doğrulama	İsteğe bağlı (Bearer JWT varsa kullanılır, Yoksa IP bazlı)	
HTTP	Durum	Yanıt Yapısı
200	OK	{ subject: string, pollen_count: int, chat_count: int, plan: "anon" "free" "premium", limits: { pollen:
500	ERROR	Veritabanı hatası

Frontend Kullanımı: UsageContext polling (usage:changed event tetiklendiğinde) → api/client.js getUsage()
→ MembershipBar kota çubuğu

3. Polen Endpointleri

GET Polen Verisi /api/pollen

Belirtilen koordinatlar için Google Pollen API'den anlık pollen verisi çeker. Önbellekte varsa (10 dk TTL, ~1 km yuvarlama) önbellekten döner. Google API başarısız olursa Open-Meteo CAMS fallback devreye girer.

Kimlik Doğrulama İsteğe bağlı (JWT varsa kullanıcı kimliği, yoksa API anahtarı) **Kota** Yok **Pollen türü** free:3, free:10, premium:sınırsız

Parametre	Yer	Tip	Zorunlu	Açıklama
lat	query	number	Evet	Enlem (-90 ile 90 arası). Örnek: 39.9334
lng	query	number	Evet	Boylam (-180 ile 180 arası). Örnek: 32.8597
days	query	integer	Hayır	Kaç günlük veri (1-5, varsayılan: 5)

HTTP	Durum	Yanıt Yapısı
200	OK	{ dailyInfo: [{ date: { year, month, day }, pollenTypeInfo: [{ code, displayName, indexInfo: { value, c
400	BAD_REQUEST	{ error, code: "INVALID_COORDS" } — geçersiz koordinat
429	QUOTA_EXCEEDED	{ error, code: "QUOTA_EXCEEDED", action: "register" "upgrade" } — günlük limit doldu
502	BAD_GATEWAY	{ error } — hem Google API hem Open-Meteo başarısız

Frontend Kullanımı: PollenContext.fetchLivePollenFor(lat, lng) → api/client.js fetchPollen() → livePollenAdapter.js adaptLivePollen() → alerts state

GET API Anahtar Durum Raporu /api/pollen/status

Google Pollen API anahtar yöneticisinin durumunu döner. Hangi anahtarların aktif, kara listede veya devre dışı olduğunu gösterir. Yalnızca yetkili kullanıcılar erişebilir.

Kimlik Doğrulama		Zorunlu (Bearer JWT)	Kota	Yok
HTTP	Durum	Yanıt Yapısı		
200	OK	{ keys: [{ index, last4, status: "active" "blacklisted" "exhausted", failCount, b		
401	UNAUTHORIZED	Geçersiz token		

Frontend Kullanımı: Admin paneli veya geliştirici araçları — normal kullanıcı akışında çağrılmaz

4. Çevre Verisi Endpointleri

GET Hava Durumu ve Hava Kalitesi /api/environment

Open-Meteo'dan hava durumu (sıcaklık, rüzgar, nem) ve hava kalitesi (AQI, PM2.5, PM10, UV) verilerini paralel çeker. Promise.allSettled kullanıldığı için biri başarısız olsa diğeri döner. API anahtarı gerektirmez.

Kimlik Doğrulama İsteğe bağlı (JWT varsa kullanıcı kimliği, yoksa PollenContext ile aynı (ortak sayaç))

Parametre	Yer	Tip	Zorunlu	Açıklama
lat	query	number	Evet	Enlem. Örnek: 41.0082
lng	query	number	Evet	Boylam. Örnek: 28.9784

HTTP	Durum	Yanıt Yapısı
200	OK	{ weather: { data: { temperature, feelsLike, humidity, windSpeed, precipitation, weatherCode, con
400	BAD_REQUEST	{ error, code: "INVALID_COORDS" }
429	QUOTA_EXCEEDED	{ error, code: "QUOTA_EXCEEDED" }

Frontend Kullanımı: PollenContext.fetchLivePollenFor() → api/client.js fetchEnvironment() → setWeather(envResult.value.weather) + setAqi(envResult.value.aqi)

Hava Kalitesi (AQI) Bant Referansı

europaAqi Aralığı	band	color (hex)	Öneri
0 - 20	Çok iyi	#4caf6e	Dışarı çıkabilirsiniz
21 - 40	İyi	#8bc34a	Normal aktivite güvenli
41 - 60	Orta	#ffc107	Hassas gruplar dikkatli olsun
61 - 80	Kötü	#ff7043	Egzersizizi kısıtlayın
81 - 100	Çok kötü	#e53935	Dışarı çıkmaktan kaçının
> 100	Tehlikeli	#7b1fa2	İç mekanda kalın

Tablo 4.1 — AQI band referansı

5. Chatbot Endpointleri

POST Chatbot Mesajı /api/chat

Kullanıcı mesajını, lokasyon bilgisini, pollen/hava/AQI özetlerini ve konuşma geçmişini alır; RAG retrieval yaparak Groq LLM'e sistem promptuyla birlikte iletir ve Türkçe yanıt döner. Groq başarısız olursa 4 model × N anahtar kombinasyonu otomatik denir.

Kimlik Doğrulama	İsteğe bağlı (JWT varsa kullanıcı kimliği, yoksa IP adresi)				Kota, yoksad, free:3, free:5, premium:sınırsız)
Parametre	Yer	Tip	Zorunlu	Açıklama	
message	body	string	Evet	Kullanıcı mesajı (max 2.000 karakter, kontrol karakterleri temizlenir)	
locationName	body	string	Evet	Seçilen konum adı (ör: "Ankara")	
lat	body	number	Evet	Enlem	
lng	body	number	Evet	Boylam	
userAllergens	body	string[]	Hayır	Kullanıcı alerjen listesi (max 30 öge × 60 karakter)	
history	body	object[]	Hayır	Konuşma geçmişi (max son 10 tur, her mesaj max 2.000 karakter). [	
clientPollen	body	string	Hayır	Frontend pollen özeti (max 2.000 karakter) — buildClientPollenSumm	
clientWeather	body	string	Hayır	weather.summary (max 500 karakter) — Open-Meteo Türkçe özet	
clientAqi	body	string	Hayır	aqi.summary (max 500 karakter) — Open-Meteo AQI Türkçe özet	
HTTP	Durum	Yanıt Yapısı			
200	OK	{ reply: string (Türkçe LLM yanıtı), model: string (kullanılan model adı) }			
400	BAD_REQUEST	{ error, code: "VALIDATION_ERROR" } — mesaj veya locationName eksik			
429	QUOTA_EXCEEDED	{ error, code: "QUOTA_EXCEEDED", action: "register" "upgrade" }			
502	LLM_UNAVAILABLE	{ error, code: "LLM_UNAVAILABLE" } — tüm Groq kombinasyonları başarısız			

Frontend Kullanımı: Chatbot.askWith(text) → api/client.js sendMessage(message, locationName, lat, lng, userAllergens, history, weather?.summary, aqi?.summary) → setMessages([...prev, { from:"bot", text: data.reply }])

Chatbot Prompt Mimarisi Özeti

Prompt Bölümü	Kaynak	Max Uzunluk
Sistem kimliği + dil kuralı	Sabit kod (chatPrompt.js)	Sabit ~200 token
Kullanıcı bilgileri (konum, alerjenler)	Request body	Değişken
RAG bilimsel chunk'ları (k=3)	Vektör deposu (RAM cosine sim.)	~500 token
Polen özeti	clientPollen (frontend → backend)	Max 2.000 karakter
Hava durumu özeti	clientWeather (frontend → backend)	Max 500 karakter
Hava kalitesi özeti	clientAqi (frontend → backend)	Max 500 karakter
Çapraz analiz kuralları	Sabit kod (chatPrompt.js)	Sabit ~150 token
LLM yanıt limiti	Groq max_tokens parametresi	512 token

Tablo 5.1 — Chatbot sistem prompt bileşenleri

6. Üyelik Endpointleri

GET Üyelik Planları /api/membership/plans

Mevcut üyelik planlarını ve fiyatlarını döner. Giriş gerektirmez.

Kimlik Doğrulama	Yok (public)	Kota	Yok
HTTP	Durum	Yanıt Yapısı	
200	OK	{ plans: [{ id: "free" "premium", name, price: number, currency: "TRY", features: string[], limits: {	

Frontend Kullanımı: UpgradeModal → api/client.js getPlans() — modal açılışında plan listesini doldurur

POST Ödeme Başlatma /api/membership/checkout

Yeni bir ödeme kaydı oluşturur (payments tablosuna "pending" satır ekler) ve paymentId döner. Frontend bu paymentId'yi iyzico ödeme sayfasına iletir.

Kimlik Doğrulama		Zorunlu (Bearer JWT)		Kota	Yok
Parametre		Yer	Tip	Zorunlu	Açıklama
plan		body	string	Hayır	"premium" (varsayılan ve tek desteklenen plan)
HTTP	Durum	Yanıt Yapısı			
201	CREATED	{ paymentId: string (ör: "pay_a1b2c3d4e5f60708"), amount: number, currency: "TRY", provider: "s			
401	UNAUTHORIZED	Geçersiz token			
409	CONFLICT	{ error } — kullanıcı zaten aktif premium			

Frontend Kullanımı: UpgradeModal → api/client.js startCheckout() → { paymentId } → iyzico ödeme akışına yönlendir → confirmMembership(paymentId, token)

POST Ödeme Doğrulama /api/membership/confirm

iyzico'dan dönen token ve daha önce oluşturulan paymentId ile ödemeyi doğrular. Sahiplik kontrolü yapılır: paymentId'nin bu kullanıcıya ait olduğu DB'den doğrulanır (IDOR koruması). Başarılıysa kullanıcı planı premium'a yükseltilir.

Kimlik Doğrulama		Zorunlu (Bearer JWT)		Kota	Yok
Parametre	Yer	Tip	Zorunlu	Açıklama	
paymentId	body	string	Evet	checkout endpoint'inden dönen paymentId	
token	body	string	Hayır	iyzico'dan dönen doğrulama token'ı (simülasyon)	
HTTP	Durum	Yanıt Yapısı			
200	OK	{ success: true, plan: "premium", plan_expires_at: string null }			
400	BAD_REQUEST	{ error } — paymentId eksik			
402	PAYMENT_FAILED	{ error, code: "PAYMENT_FAILED" } — ödeme başarısız veya token geçersiz			
403	FORBIDDEN	{ error, code: "FORBIDDEN" } — paymentId bu kullanıcıya ait değil (IDOR)			
401	UNAUTHORIZED	Geçersiz token			

Frontend Kullanımı: UpgradeModal → api/client.js confirmMembership(paymentId, iyzicoToken) → AuthContext.refetchUser() → plan güncellendi

POST**Premium İptal** </api/membership/cancel>

Kullanıcının premium üyeliğini iptal eder. Hesap ücretsiz plana düşürülür.

Kimlik Doğrulama	Zorunlu (Bearer JWT)	Kota	Yok
HTTP	Durum	Yanıt Yapısı	
200	OK	{ success: true, plan: "free" }	
401	UNAUTHORIZED	Geçersiz token	
409	CONFLICT	{ error } — kullanıcı zaten free planda	

Frontend Kullanımı: AccountMenu "Üyeliği İptal Et" → `api/client.js cancelMembership()` → `AuthContext.refetchUser()`

7. Sistem Endpointleri

GET Sağlık Kontrolü /api/health

Sunucu ve veritabanı bağlantısının çalışıp çalışmadığını kontrol eder. Yük dengeleyici (load balancer) ve izleme araçları tarafından kullanılır. Rate limit uygulanmaz.

Kimlik Doğrulama	Yok (public)	Kota	Yok (rate limit muaf)
HTTP	Durum	Yanıt Yapısı	
200	OK	{ status: "ok", database: true }	
503	UNAVAILABLE	{ status: "ok", database: false } — DB'ye erişilemiyor	

Frontend Kullanımı: Kubernetes readiness probe / uptime monitoring — frontend tarafından doğrudan çağrılmaz

GET Kök Endpoint /

API'nin çalışır durumda olduğunu doğrular. Web arayüzü sunmaz.

Kimlik Doğrulama	Yok (public)	Kota	Yok
HTTP	Durum	Yanıt Yapısı	
200	OK	{ status: "ok", service: "Alerji Takip API", hint: "Bu bir API sunucusudur" }	

Frontend Kullanımı: Geliştirici doğrulama amacıyla kullanılır

8. Frontend Entegrasyon Özeti

Bu bölüm, frontend (api/client.js) ile backend arasındaki tüm bağlantıları tek tabloda özetler. Her satır bir API fonksiyonunu ve hangi React bileşeninin onu çağırdığını gösterir.

Frontend Fonksiyon (api/client.js)	Frontend Bileşen	Backend Endpoint	Tetikleyici
login(email, password)	AuthModal	POST /api/login	Form submit
register(name, email, password)	AuthModal	POST /api/register	Form submit
getMe()	AuthContext	GET /api/me	Uygulama yüklenişi, token yenileme
updateProfile(profileData)	DashboardProfilePage	PUT /api/profile	Kaydet butonu
updateFavorites(favorites[])	MapSelector	PUT /api/favorites	Favori ekle/çıkar
deleteAccount(password)	AccountMenu	DELETE /api/account	Hesabı sil + onay
getUsage()	UsageContext	GET /api/usage	usage:changed event dinleyicisi
fetchPollen(lat, lng, days)	PollenContext	GET /api/pollen	Konum değişimi
fetchEnvironment(lat, lng)	PollenContext	GET /api/environment	Konum değişimi (paralel)
sendChatMessage(msg, loc, lat, lng, to, from, history, readStatus)	ChatModal	POST /api/chat	Mesaj gönder (Enter / buton)
getPlans()	UpgradeModal	GET /api/membership/plans	Modal açılışı
startCheckout()	UpgradeModal	POST /api/membership/checkout	"Beklemlerim Al" butonu
confirmMembership(paymentId, token)	UpgradeModal	POST /api/membership/confirm	confirm callback
cancelMembership()	AccountMenu	POST /api/membership/cancel	"İptal Et" butonu
logout()	AccountMenu	(localStorage)	"Çıkış Yap" butonu — endpoint yok

Tablo 8.1 — Frontend → Backend bağlantı matrisi

Global Olay (Event) Sistemi

api/client.js, hata durumlarında CustomEvent fırlatır. Bu sayede bileşenler doğrudan birbirini import etmeden iletişim kurar (decoupled mimari):

Event Adı	Nerede Fırlatılır	Dinleyen Bileşen	Aksiyon
auth:unauthorized	401 hatası (login/register dışında)	AuthContext	localStorage token temizle, kullanıcı state sıfırla
quota:exceeded	429 + code:QUOTA_EXCEEDED	UsageContext	UpgradeModal aç, kota çubuğunu güncelle
usage:changed	fetchPollen() ve sendChatMessage()	UsageContext	getUsage() yeniden çağır, kota sayaçlarını güncelle

Tablo 8.2 — Global CustomEvent sistemi